Practical 5

AIM:

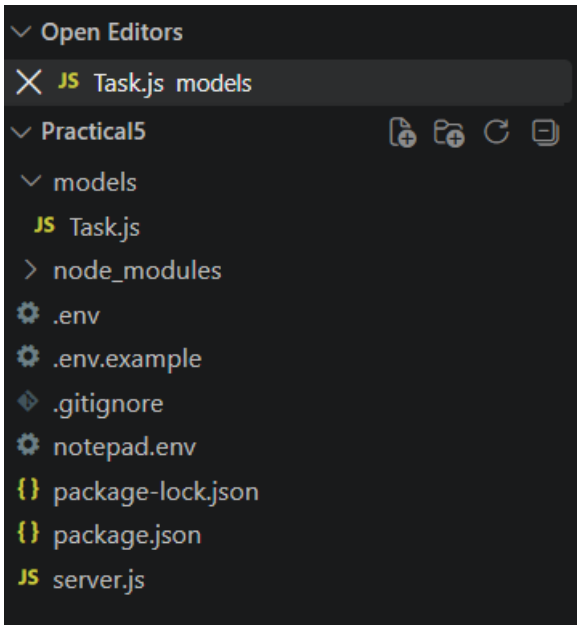
• Extend the Task Management backend from Practical 4 by connecting MongoDB using Mongoose.

• Define a Task schema with at least 4 fields: title (String, required), description (String), completed (Boolean, default false), createdAt (Date, default Date.now).

• Replace the in-memory array from Practical 4 with real Mongoose model operations.

• Test all CRUD operations against the live database using Postman. • Ensure validation errors

Folder Structure:



server.js

```
/**
 * Practical 5 – MongoDB Integration and Schema Design with Mongoose
 *
 * Objective: Replace the in-memory array from Practical 4 with a real
```

```
 * MongoDB database accessed through the Mongoose ODM.
 *
 * ── Architecture ──────────────────────────────────────────────────────
 *   Express App
 *     └── Mongoose ODM
 *           └── MongoDB Database
 *                 └── tasks collection
 *                       { title, description, completed, priority, createdAt }
 *
 * ── What changed from Practical 4 ─────────────────────────────────────
 *   - Added dotenv to keep MONGO_URI out of source code
 *   - Added mongoose.connect() before starting the server
 *   - Replaced in-memory array operations with Mongoose model methods:
 *       Task.find()            ← was: tasks array
 *       Task.create()          ← was: tasks.push()
 *       Task.findByIdAndUpdate() ← was: tasks.find() + mutate
 *       Task.findByIdAndDelete() ← was: tasks.splice()
 *   - Added GET /tasks/:id endpoint with proper 404 handling (supplementary)
 *   - Validation errors from Mongoose are caught and returned as clean JSON
 *     (not the raw Mongoose ValidationError object)
 *   - Every route is wrapped in try/catch and passes errors to next(err)
 *
 * ── Setup ─────────────────────────────────────────────────────────────
 –
```

```
 *   1. Copy .env.example to .env and set MONGO_URI
 *   2. npm install
 *   3. node server.js  (or: npm run dev)
 *   4. Test with Postman on http://localhost:5000
 *
 * ── Routes ──────────────────────────────────────────────────────────
 *   GET    /tasks        → all tasks
 *   GET    /tasks/:id    → single task (404 if not found)
 *   POST   /tasks        → create task  (validates title)
 *   PUT    /tasks/:id    → update task  (404 if not found)
 *   DELETE /tasks/:id    → delete task  (404 if not found)
 */

const express  = require('express')
const mongoose = require('mongoose')
require('dotenv').config()         // loads .env into process.env

const Task = require('./models/Task')

const app  = express()
const PORT = process.env.PORT || 5000

/* ─────────────────────────────────────────────────
   Middleware Pipeline
──────────────────────────────────────────────────── */
```

```
// 1. Parse JSON bodies
app.use(express.json())

// 2. Request logger
app.use((req, res, next) => {
  console.log(`[${new Date().toISOString()}] ${req.method} ${req.url}`)
  next()
})

// 3. Content-Type check for write operations
app.use((req, res, next) => {
  if (
    (req.method === 'POST' || req.method === 'PUT') &&
    req.headers['content-type'] !== 'application/json'
  ) {
    return res.status(400).json({
      error: 'Content-Type must be application/json for POST and PUT requests',
    })
  }
  next()
})

/* ————————————————————————————————————
   Helper: format Mongoose validation errors
   into a clean object instead of sending the
   raw ValidationError to the client.
```

```
──────────────────────────────────────────────────────────── */
function formatValidationError(err) {
  const errors = {}
  Object.keys(err.errors).forEach((field) => {
    errors[field] = err.errors[field].message
  })
  return errors
}

/* ────────────────────────────────────────────────────────────
   CRUD Routes
──────────────────────────────────────────────────────────── */

// GET /tasks – return all tasks
app.get('/tasks', async (req, res, next) => {
  try {
    const tasks = await Task.find().sort({ createdAt: -1 })
    res.status(200).json(tasks)
  } catch (err) {
    next(err)
  }
})

// GET /tasks/:id – return a single task (supplementary)
app.get('/tasks/:id', async (req, res, next) => {
  try {
```

```
    const task = await Task.findById(req.params.id)

    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.status(200).json(task)
  } catch (err) {
    // CastError means the :id was not a valid MongoDB ObjectId
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

// POST /tasks – create a new task
app.post('/tasks', async (req, res, next) => {
  try {
    const task = await Task.create({
      title:       req.body.title,
      description: req.body.description,
      completed:   req.body.completed,
      priority:    req.body.priority,
    })
```

```
    res.status(201).json(task)
  } catch (err) {
    // Mongoose validation error → return structured field-level messages
    if (err.name === 'ValidationError') {
      return res.status(400).json({ errors: formatValidationError(err) })
    }
    next(err)
  }
})

// PUT /tasks/:id – update an existing task
app.put('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndUpdate(
      req.params.id,
      {
        title:       req.body.title,
        description: req.body.description,
        completed:   req.body.completed,
        priority:    req.body.priority,
      },
      {
        new:         true,   // return the updated document, not the old one
        runValidators: true,  // run schema validators on update as well
      }
    )
```

```
    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }

    res.status(200).json(task)
  } catch (err) {
    if (err.name === 'ValidationError') {
      return res.status(400).json({ errors: formatValidationError(err) })
    }
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

// DELETE /tasks/:id – delete a task
app.delete('/tasks/:id', async (req, res, next) => {
  try {
    const task = await Task.findByIdAndDelete(req.params.id)

    if (!task) {
      return res.status(404).json({ error: 'Task not found' })
    }
```

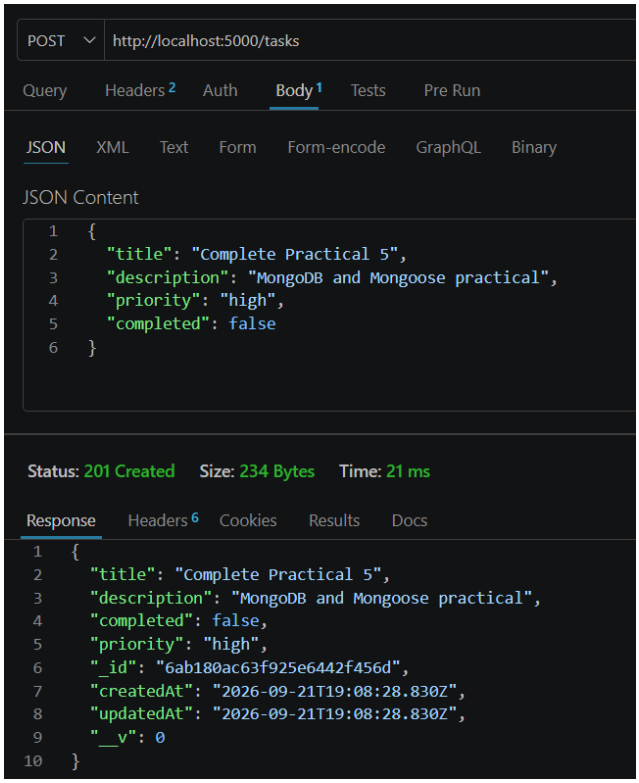
```
    res.status(200).json({ message: 'Task deleted successfully', task })
  } catch (err) {
    if (err.name === 'CastError') {
      return res.status(400).json({ error: 'Invalid task ID format' })
    }
    next(err)
  }
})

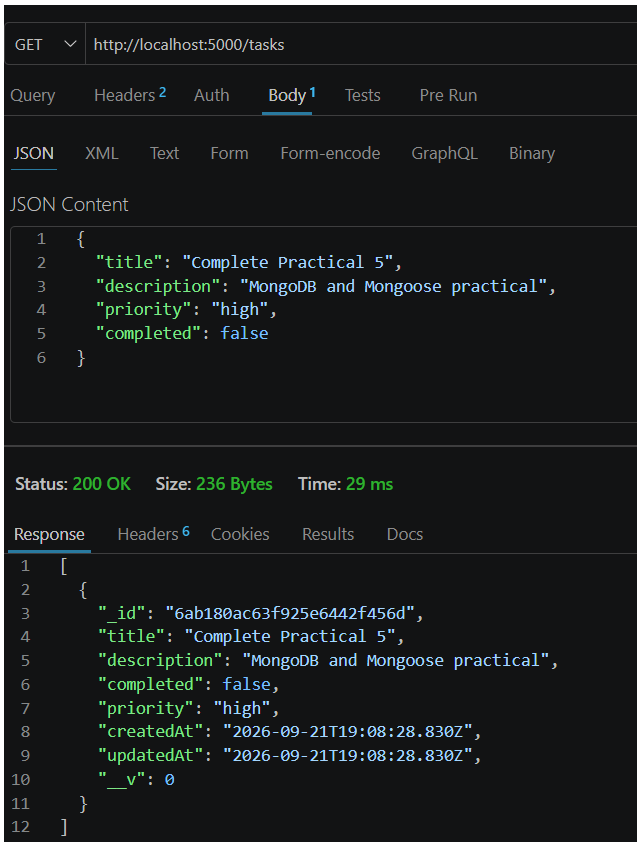
/* ————————————————————————————————————
   404 Handler – undefined routes
———————————————————————————————————— */
app.use((req, res) => {
  res.status(404).json({ error: `Route ${req.method} ${req.url} not found` })
})

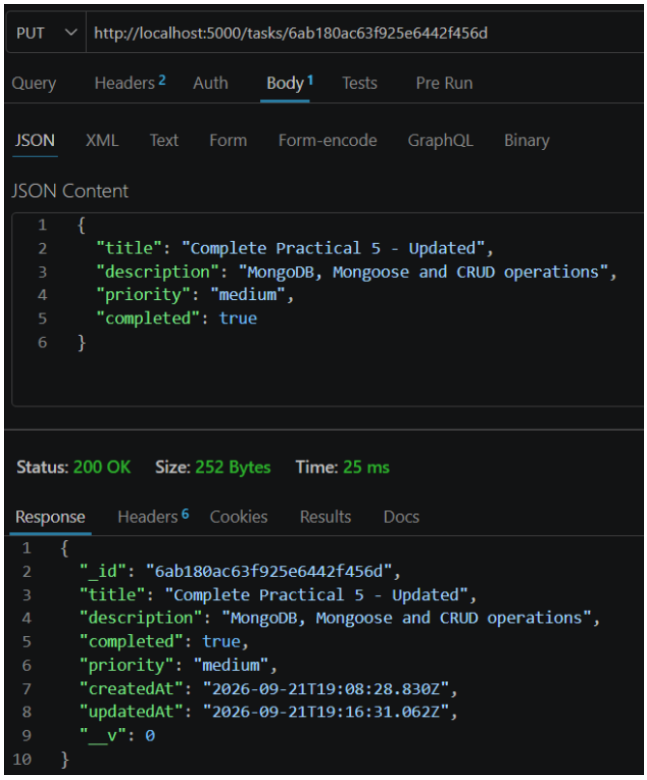
/* ————————————————————————————————————
   Global Error Handler (must be last, 4 params)
———————————————————————————————————— */
app.use((err, req, res, next) => {
  console.error(`[ERROR] ${err.stack}`)
  res.status(500).json({ error: 'Something went wrong on the server' })
})

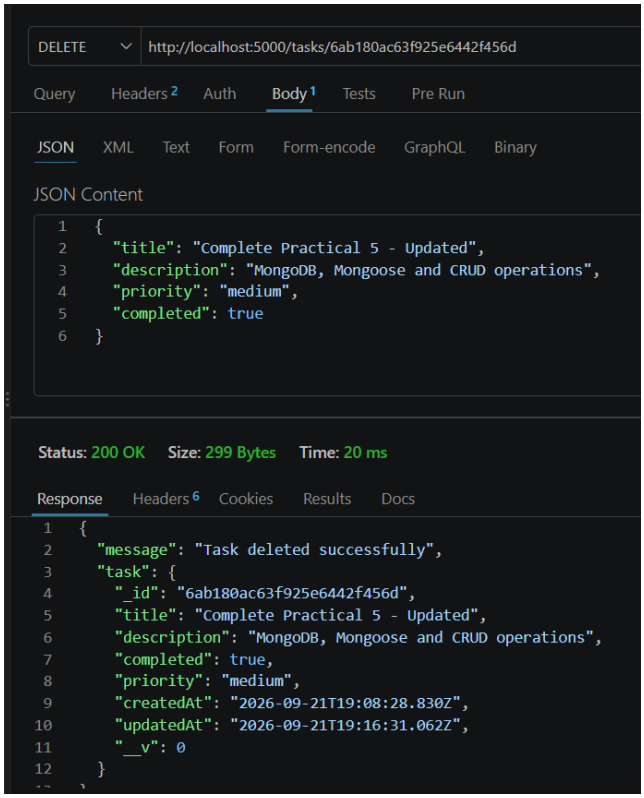
/* ————————————————————————————————————
   Connect to MongoDB, then start the server.
```

```
   Server only starts AFTER a successful DB
   connection — avoids accepting requests before
   the database is ready.
──────────────────────────────────────────────────── */
mongoose
  .connect(process.env.MONGO_URI)
  .then(() => {
    console.log('MongoDB connected successfully')
    app.listen(PORT, () => {
      console.log(`Server running on http://localhost:${PORT}`)
      console.log('Available routes:')
      console.log('  GET    /tasks')
      console.log('  GET    /tasks/:id')
      console.log('  POST   /tasks')
      console.log('  PUT    /tasks/:id')
      console.log('  DELETE /tasks/:id')
    })
  })
  .catch((err) => {
    console.error('MongoDB connection failed:', err.message)
    console.error('Make sure your MONGO_URI in .env is correct.')
    process.exit(1)
  })
```


POST
http://localhost:5000/tasks
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"priority": "high",
"completed": false
}
Status: 201 Created
Size: 234 Bytes
Time: 21 ms
Response
Headers 6
Cookies
Results
Docs
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"completed": false,
"priority": "high",
"_id": "6ab180ac63f925e6442f456d",
"createdAt": "2026-09-21T19:08:28.830Z",
"updatedAt": "2026-09-21T19:08:28.830Z",
"__v": 0
}


GET
http://localhost:5000/tasks
Query
Headers 2
Auth
Body 1
Tests
Pre Run
JSON
XML
Text
Form
Form-encode
GraphQL
Binary
JSON Content
{
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"priority": "high",
"completed": false
}
Status: 200 OK
Size: 236 Bytes
Time: 29 ms
Response
Headers 6
Cookies
Results
Docs
[
{
"_id": "6ab180ac63f925e6442f456d",
"title": "Complete Practical 5",
"description": "MongoDB and Mongoose practical",
"completed": false,
"priority": "high",
"createdAt": "2026-09-21T19:08:28.830Z",
"updatedAt": "2026-09-21T19:08:28.830Z",
"__v": 0
}
]





.env

MONGO_URI=mongodb://127.0.0.1:27017/Practical5

.env.example

```
MONGO_URI=mongodb://localhost:27017/taskdb
PORT=5000
```

package.json

```
{
  "name": "practical-5-mongodb-mongoose",
  "version": "1.0.0",
  "description": "Practical 5 – MongoDB Integration and Schema Design with Mongoose",
  "main": "server.js",
  "type": "commonjs",
  "scripts": {
    "start": "node server.js",
    "dev": "nodemon server.js"
  },
  "dependencies": {
    "cors": "^2.8.6",
    "dotenv": "^16.3.1",
    "express": "^4.18.2",
    "mongoose": "^8.0.3"
  },
  "devDependencies": {
    "nodemon": "^3.0.1"
  }
}
```

.gitignore

```
node_modules/
.env
```

Supplementary Problem:

• Add a priority field to the schema restricted to an enum of ['low', 'medium', 'high'].

```
priority: {
   type: String,
   enum: ['low', 'medium', 'high'],
   default: 'medium'
}
```

• Add a pre-save hook that automatically trims whitespace from the title field.

Task.js

```
const mongoose = require('mongoose');

const taskSchema = new mongoose.Schema(
   {
      title: {
         type: String,
         required: [true, 'Title is required'],
         trim: true
      },

      description: {
         type: String,
         trim: true
      },

      completed: {
```

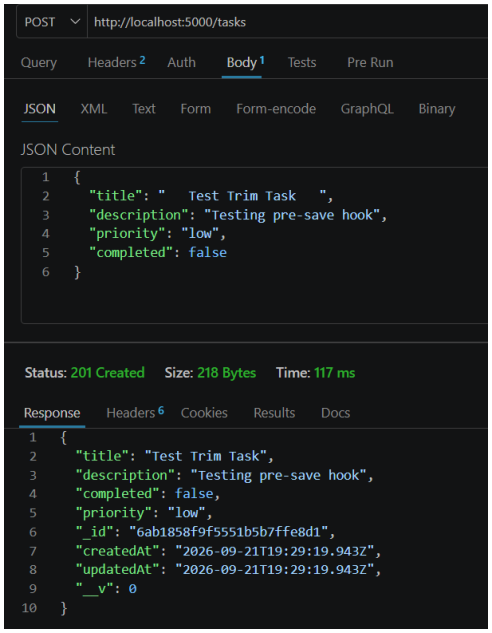
```
        type: Boolean,
        default: false
      },

      priority: {
        type: String,
        enum: ['low', 'medium', 'high'],
        default: 'medium'
      }
    },
    {
      timestamps: true
    }
);

// Pre-save hook: trim whitespace from title
taskSchema.pre('save', function (next) {
    if (this.isModified('title') && this.title) {
        this.title = this.title.trim();
    }
    next();
});

const Task = mongoose.model('Task', taskSchema);

module.exports = Task;
```



• Implement a GET /tasks/:id endpoint that returns a 404 JSON response if the ID does not exist.

Server.js

```
// GET /tasks/:id - Get a single task by ID
app.get('/tasks/:id', async (req, res, next) => {
  try {
    const { id } = req.params;

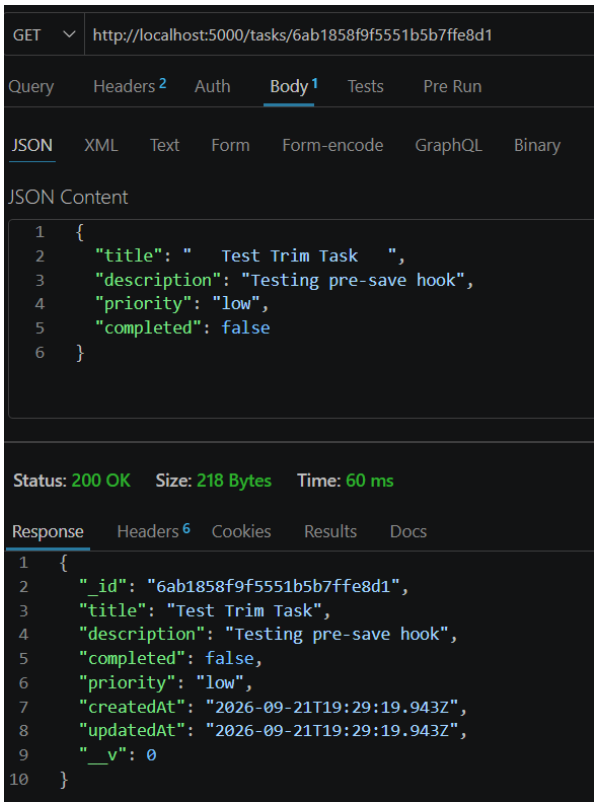
    // Check whether the ID is a valid MongoDB ObjectId
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({
        error: 'Invalid task ID'
      });
    }

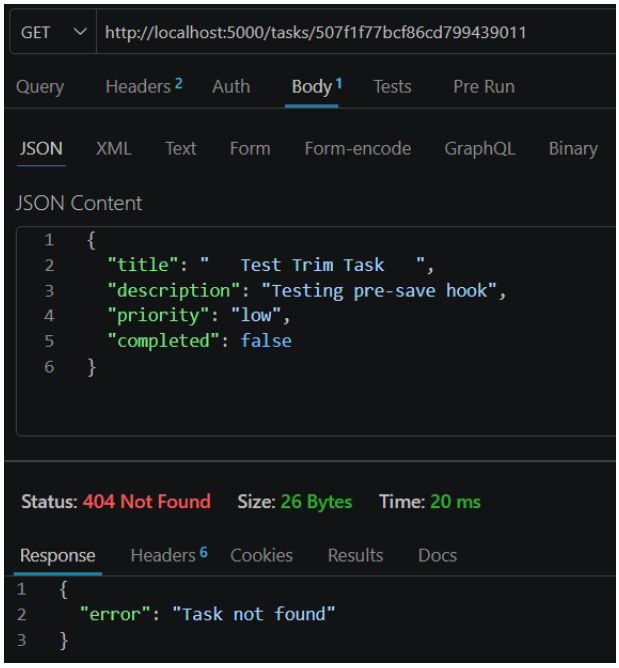
    const task = await Task.findById(id);
```

```
        // Task ID is valid but task does not exist
        if (!task) {
            return res.status(404).json({
                error: 'Task not found'
            });
        }

        // Task found
        res.status(200).json(task);

    } catch (err) {
        next(err);
    }
});
```





Github Repo: https://github.com/Bhumi3009/D25DIT090_ITUE301_AWDF